

Templates; MiniVector Interface & Memory Management

CSCI 235 — Class #3

Tong Yi

Hunter College, CUNY

Tuesday, September 8, 2026

From Your Lecture Slips

Answers to the questions you turned in last class, grouped by topic.

The Ternary Operator ?:

Your question

“How/why do we use the operator ? and what benefits does it have?”

```
condition ? valueIfTrue : valueIfFalse
```

Why it's useful

Unlike `if/else` (a *statement*), `?:` is an *expression* — it fits anywhere an expression is expected, including inside a member initializer list:

```
capacity_(count > 0 ? count : 2) // fits right in
```

`if/else` can't go inside a member initializer — use `?:` for a simple choice there, or move more complicated branching logic into the constructor *body*.

Why Use a Member Initializer List?

Your question

“Why do we use a member initializer list?”

- A member initializer list is **required** for const members and reference members, and for specifying how a base-class subobject is constructed — these initializations happen *before* the constructor body runs.
- For a member that is itself a class type, the initializer list constructs it directly with the right constructor. Without it, the member is default-constructed first, then *reassigned* in the body — wasted work, and a compile error if that type has no default constructor.
- As we saw today: initializer-list entries run in **declaration order**, no matter what order you write them in.

The `this` Pointer and the `::` Operator

Your question

“Could you explain the `this` keyword and the `::` symbol?”

`this`

Every non-static member function secretly receives a pointer to the object it was called on, named `this`. Inside a `const` member function, `this` has type `const ClassName*` — it's how the function knows *which* object's data to use.

`::` (scope resolution)

Qualifies a name with the scope it belongs to. Defining a member function outside the class body needs it:

```
double Rectangle::area() const { return length_ * width_; }  
// "this area() belongs to class Rectangle"
```

The Colon Before a Constructor Body

Your question

“Single colon syntax? I’ve never seen that.”

You’ve actually seen it in every constructor today:

```
ClassName::ClassName(params)
    : member1(value1), member2(value2) {
    // body (may be empty)
}
```

What it is

The single `:` after the parameter list starts the **member initializer list** — a comma-separated sequence of `member(value)` entries, run *before* the body, in declaration order. If a constructor needs no initializer list, the colon (and everything up to `{`) is simply omitted.

When Do We Use operator=?

Your question

“When do we want to use operator= ?”

```
Rectangle a, b(3, 4);  
a = b;           // a already exists -> operator=  
Rectangle c = b; // c is being created -> copy constructor, not  
                operator=
```

Rule of thumb

operator= runs only when the object on the left of = already exists. If the object is being created in the same statement, that's construction, not assignment — even though both use the = symbol. In `Rectangle c = b;` specifically, the copy constructor is used.

Direct- vs. Copy-Initialization

Your question

“I’d like to go over the direct & copy initializations a bit more.”

```
// Two alternative ways to create r4:  
Rectangle r4(r3);    // direct-initialization  
// Rectangle r4 = r3; // copy-initialization
```

The key point

Either line *constructs* `r4` using the **copy constructor** — `r4` doesn’t exist yet, so `=` here is *not* assignment, despite appearances. The two forms differ in initialization *syntax*, not in which special member function runs.

What Does “Overload” Mean?

Your question

“What are we creating with this code? Also, what does overload mean?”

Overloading

Two or more functions/methods share the **same name** but different **parameter lists** (number, order, or type); return type alone never distinguishes an overload. Operators can be overloaded too: `operator=`, `operator[]`.

Not *override*

Override: same method name, same parameters, same const qualification. Overload: same name, **different** parameter list — no inheritance needed.

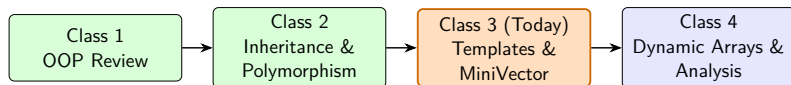
Logistics Recap

- **Extra credit:** none.
- **Dropped scores:** no project is dropped; the **lowest of two** (out of 11) exercise scores is dropped.
- **Gradescope course code:** V4N6ZP.
- **Pop quizzes:** content and grading — *to be announced in class.*

Also noted

Several slips asked that the microphone be used — several students said it was hard to hear otherwise.

Where We Are



Today's outline:

- 1 Why templates exist; function templates
- 2 Stack vs. heap, pointers, new/delete
- 3 Designing MiniVector: interface & memory management

The Problem: Code That Repeats Itself

Suppose we want a max function. Without templates, we would need a separate copy for every type we care about:

```
int maxInt(int a, int b) {  
    return (a > b) ? a : b;  
}  
  
double maxDouble(double a, double b) {  
    return (a > b) ? a : b;  
}  
  
char maxChar(char a, char b) {  
    return (a > b) ? a : b;  
}
```

The Problem: Code That Repeats Itself

Suppose we want a max function. Without templates, we would need a separate copy for every type we care about:

```
int maxInt(int a, int b) {  
    return (a > b) ? a : b;  
}  
  
double maxDouble(double a, double b) {  
    return (a > b) ? a : b;  
}  
  
char maxChar(char a, char b) {  
    return (a > b) ? a : b;  
}
```

The issue

The *logic* is identical in every version. Only the *type* changes. Copy-pasting logic is a maintenance trap: fix a bug in one copy, and you must remember to fix it in all the others.

Function Templates

A **template** lets us write the logic once, with the type left as a blank to be filled in later:

```
template <typename T>
T myMax(T a, T b) {
    return (a > b) ? a : b;
}
```

```
myMax(3, 7);           // T = int,   returns 7
myMax(3.5, 2.1);       // T = double, returns 3.5
myMax('a', 'z');       // T = char,  returns 'z'
```

Intuition

Think of `template <typename T>` as a fill-in-the-blank recipe. The compiler reads your call, sees what type you plugged in, and *stamps out* a fresh, ordinary function for that type — as if you had written `maxInt`, `maxDouble`, and `maxChar` by hand, but without you actually typing them.

Another Example: swap

```
template <typename T>
void mySwap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
```

- T& means “a reference to whatever type T turns out to be” — mySwap modifies the caller’s actual variables, just like a non-template reference parameter would.
- T temp = a; declares a local variable of type T to hold a temporarily while we overwrite it.
- This single template works for int, double, std::string, or any type that supports copying — no rewriting required.

Class Templates (a Preview)

Templates aren't limited to functions — an entire *class* can be templated on a type:

```
template <typename T>
class Box {
    T value;
public:
    Box(T v) : value(v) {}
    T get() const { return value; }
};
Box<int> b1(42);
```

This is exactly how `std::vector<T>` works

`std::vector<int>`, `std::vector<double>`, ... are all the *same* class template, instantiated for different `T`.

MiniVector is deliberately *not* templated — only ever `int`. More on class templates later in the course.

Two Kinds of Memory

Stack memory

Local variables, function parameters

Automatic lifetime

Destroyed when their scope ends

Heap memory

Allocated dynamically with `new` / `new[]`

Freed explicitly with `delete` / `delete[]`

Forget to delete?
A memory leak.

Analogy

Stack = cafeteria trays: one appears when you need it, cleared away automatically when you're done. **Heap** = a self-storage unit: you must rent it (`new`) and return it (`delete`) — or keep paying forever.

Pointers, new, and delete

```
int* p = new int(42); // one int on the heap, init to 42
delete p;             // free it -- exactly one delete per new
p = nullptr;         // avoid a dangling pointer

int* arr = new int[10]; // an array of 10 ints on the heap
delete[] arr;           // must use delete[] for arrays
arr = nullptr;         // avoid a dangling pointer
```

Rule of thumb

Every successful allocation with `new` must eventually have a matching `delete`; every successful `new[]` must eventually have a matching `delete[]`. Mixing them is undefined behavior.

- `p` lives on the *stack*; what it *points to* lives on the *heap*.
- After `delete`, `p` still contains a **dangling pointer**.
Setting it to `nullptr` makes the invalid state explicit and lets us check it before use.

Null Pointers

A pointer that doesn't point to any valid object should be set to `nullptr`:

```
int* p = nullptr; // p points to "nothing"
std::cout << *p;  // ERROR: undefined behavior
```

Why this matters

Dereferencing (`*p` or `p->member`) a null pointer causes **undefined behavior** and often crashes the program. Before dereferencing a pointer that *might* be null, check it first:

```
if (p != nullptr) {
    std::cout << *p;
}
```

`nullptr` is also the natural “I have no heap memory yet” value for a pointer data member before its first `new`.

Memory Leaks

```
void leaky() {  
    int* data = new int[1000];  
    // ... use data ...  
} // data is gone, but the 1000-int heap array is never freed!
```

- The *pointer* data is a stack variable — it disappears automatically when leaky() returns.
- The *array* it pointed to is heap memory — it does **not** clean itself up, and once unreachable, that memory is wasted for the rest of the program.
- Call leaky() in a loop, and you leak a little more memory every time.

The fix we're about to build

Wrap the allocation inside a *class* whose **destructor** calls `delete[]` automatically — exactly what `MiniVector`'s destructor will do.

What Is MiniVector?

MiniVector is a simplified, `int`-only version of `std::vector<int>`, built from raw heap memory. You already use `std::vector` without thinking about how it grows; this project makes you build that growth machinery yourself.

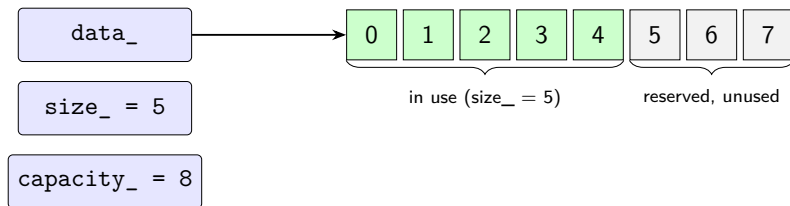
Why build it from scratch?

`std::vector` hides three ideas behind a clean interface:

- how a dynamically-sized array is actually stored on the heap,
- how it *grows* when it runs out of room,
- what it means to safely *copy* an object that owns heap memory.

MiniVector makes all three visible.

MiniVector's Data Members



```
int* data_;           // pointer to a dynamically allocated heap array
std::size_t size_;    // logical number of elements in use
std::size_t capacity_; // physical array size before reallocation
```

Invariant: `size_ <= capacity_` always holds.

Example: What's Wrong Here?

Here is an attempted count constructor for MiniVector:

```
MiniVector::MiniVector(std::size_t count)
    : capacity_(count > 0 ? count : 2),
      size_(capacity_),
      data_(new int[capacity_]{} ) {}
```

Take a minute

This *compiles*, and often *seems* to work. There are two separate bugs in it — one about *when* things happen, one about *what value* gets stored. Can you find both?

Bug 1: Initializer Lists Run in Declaration Order

Recall how MiniVector's data members are *declared* in the class:

```
int* data_;           // declared 1st
std::size_t size_;    // declared 2nd
std::size_t capacity_; // declared 3rd
```

The rule

C++ initializes members in **declaration order**. The order written in the initializer list does not control initialization order — declaration order *is* the initialization order.

So the real order is `data_`, `size_`, `capacity_` — both `size_(capacity_)` and `data_(new int[capacity_]{} )` read `capacity_` *before it has been initialized*, so the program has **undefined behavior**: the resulting array size is unpredictable and may lead to a crash or `std::bad_alloc`.
-Wall also warns about this via `-Wreorder`.

Bug 2: `size_` Isn't Always `capacity_`

Even ignoring Bug 1, `size_(capacity_)` is only correct by *coincidence*.

- Recall the spec: for `count == 0`, the constructor must behave exactly like the default constructor — `capacity_ = 2, size_ = 0`.
- This code *does* set `capacity_ = 2` when `count == 0` — correct. But it also sets `size_ = capacity_ = 2` — **wrong**; it should be 0.

Why this matters

A `MiniVector` built with `count == 0` would incorrectly report 2 elements already “in use.” `size_` (elements logically present) and `capacity_` (room reserved) answer different questions — conflating them breaks exactly the case that reveals the difference.

A Correct Version: Declaration-Order_INITIALIZER List

```
MiniVector::MiniVector(std::size_t count)
    : data_(new int[count > 0 ? count : 2]{}),
      size_(count),
      capacity_(count > 0 ? count : 2) {
}
```

Why this works

Written in **declaration order** (`data_`, `size_`, `capacity_`), and no initializer references another member — each depends only on the parameter `count`.

A nice simplification

`size_(count)` alone already handles `count == 0` correctly: when `count` is 0, `size_` becomes 0 too. No ternary needed there.

Also Correct: Compute in the Body

For these particular data members, this is also correct — the constructor *body* runs in the order *written*:

```
MiniVector::MiniVector(std::size_t count) {  
    capacity_ = (count > 0 ? count : 2);  
    size_     = count;  
    data_     = new int[capacity_]{};  
}
```

Rule of thumb

Prefer the initializer list, in declaration order, with each initializer depending only on the constructor's parameters. Reach for the body only when that's not possible.

MiniVector's Interface, Task by Task

Task	Functions	Big idea
A	constructors, destructor	own the heap array
B	reserve(), push_back()	grow the array
C	pop_back(), clear(), at()	change logical size; bounds checking
D	copy constructor, operator=	copy an owner of heap memory

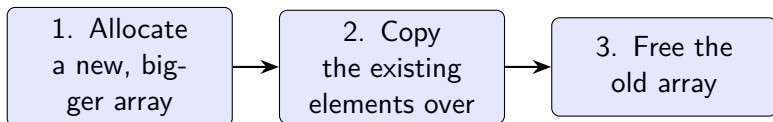
Plus, already provided: `operator[]`, `empty()`, `size()`, `capacity()`.

Today's focus

We'll walk through the *ideas* behind Tasks A–C: how the array is owned and how it grows. You'll write the code yourselves.

`reserve()`: **Allocate** — **Copy** — **Free**

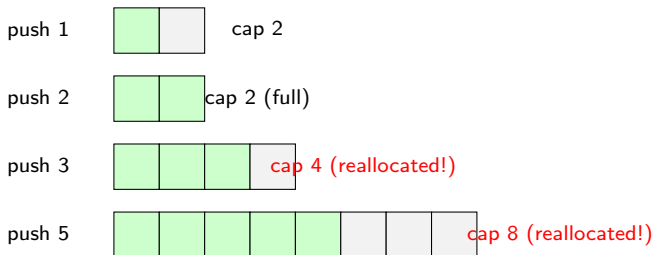
When more room is needed, `reserve(newCapacity)` follows the same three-step pattern every time:



- If `newCapacity` is not larger than the current `capacity()`, `reserve()` does nothing — capacity never shrinks.
- This “allocate–copy–free” pattern is the *only* place reallocation happens. Every other function that needs more room calls `reserve()` rather than reimplementing this logic.

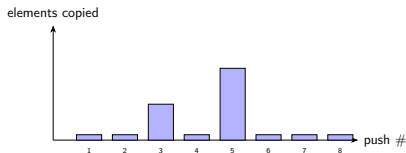
push_back(): Growing by Doubling

When the array is full, `push_back` calls `reserve(2 * capacity())` *before* inserting the new element. (*Selected pushes shown below — see all eight capacities underneath.*)



Capacity after each of the first 8 pushes: 2, 2, 4, 4, 8,
8, 8, 8

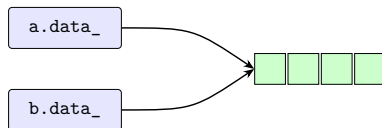
Why Doubling? Amortized $O(1)$ push_back



- Most calls to `push_back` are cheap: write one element, increment `size_`. Constant time.
- Occasionally a reallocation happens, copying every existing element — expensive, but *increasingly rare*.
- Capacity *doubles*, so each reallocation copies roughly twice as many *old* elements as before: across n calls, the total copied due to reallocation is $2 + 4 + 8 + \dots < 2n$ — still $O(n)$, separate from each push's own $O(1)$ write.
- Spread that $O(n)$ over n calls: $O(n)/n = O(1)$ **amortized** per `push_back`.

- `pop_back()` decrements `size_` by one. It does *not* touch `capacity_` or reallocate — shrinking the logical size is cheap.
 - Precondition: not called on an empty `MiniVector` (undefined behavior otherwise, just like `std::vector`).
- `clear()` sets `size_` to 0 — same idea, just all at once. The underlying array and its capacity are untouched.
- `at(idx)` is the *safe* sibling of `operator[]`: it checks `idx` against `size()` and throws `std::out_of_range` if it's out of bounds. `operator[]` makes no such promise — out-of-range access is undefined behavior, exactly like a raw array.

A Preview: Why Copying Needs Care (Task D)



The danger of a shallow copy

Without a correctly implemented copy constructor, a shallow copy would copy `data_` *itself* — so `a` and `b` would point at the *same* heap array, as drawn above:

- Modifying `b` silently changes `a` too.
- When both destructors run, the array gets `delete[]`'d *twice* — undefined behavior (a “double free”).

Task D's copy constructor and `operator=` must instead perform a **deep** copy — a separate array with its own copy of the elements. More soon.

Common Pitfalls

- **Mismatched new/delete:** using `delete` where `delete[]` was needed (or vice versa).
- **Forgetting to free the old array** inside `reserve()` before overwriting `data_` — a leak on every reallocation.
- **Dereferencing before checking for null** — guard with `if (p != nullptr)`.
- **Off-by-one on capacity:** writing to index `size_` without first confirming `size_ < capacity()`.
- **Shallow copies** of a class that owns heap memory — the subject of Task D.

What's Due

Item	Posted	Due
Exercise 2 (Pointer Practice)	today, Tu 9/8	Tu 9/15
Exercise 1 (Shape, Triangle, Square)	Th 9/3	Th 9/10
Project 1A: MiniVector	Tu 9/1	Th 9/17

Full details and due dates are always on the course schedule page.

Note

No extra credit is offered this semester.

Key Takeaways

- **Templates** let you write one generic version of a function or class instead of duplicating logic per type.
- **Stack** memory is automatic; **heap** memory (`new/delete`) is your responsibility.
- Every successful `new` must eventually have a matching `delete`; every `new[]` needs a matching `delete[]`.
- `MiniVector`'s `reserve()` always follows **allocate** → **copy** → **free**.
- Doubling capacity gives `push_back` **amortized** $O(1)$ time.
- A class that owns heap memory needs **deep** copies — coming up in Task D.

Questions?

Office hours and the course schedule are on the course website.

Backup Slides

More attempts at the count constructor — use these only if students propose exactly these solutions.

More Attempts: Does This Even Compile?

```
MiniVector::MiniVector(std::size_t count) :  
    count > 0  
    ? size_(count), capacity_(count), data_(new int[count]{} )  
    : size(0), capacity_(2), data_(new int[2]{} )  
    {}
```

No — this isn't valid C++

An initializer list is a comma-separated list of member(args) entries; ?: cannot choose between two whole *groups* of them. The compiler tries to read count itself as a member name and rejects it immediately:

```
error: class 'MiniVector' does not have any field named 'count'  
error: expected '(', before '>' token
```

(There's also an unrelated typo: size(0) is missing its underscore.)

Fixed the Value, Not the Order

```
capacity_(count > 0 ? count : 2),  
size_(count > 0 ? capacity_ : 0),  
data_(new int[capacity_]{})
```

`size_`'s *formula* is now correct — it matches the Bug 2 fix. But `size_` and `data_` still read `capacity_` *before it exists*: declaration order (`data_`, `size_`, `capacity_`) is unchanged. -Wall confirms it:

```
warning: 'capacity_' will be initialized after 'size_' [-Wreorder]  
warning: 'size_' will be initialized after 'data_' [-Wreorder]  
warning: '*this.capacity_' is used uninitialized [-Wuninitialized]
```

Caveat: -Wuninitialized only caught the `data_` case here — it missed `size_`'s use of `capacity_` inside the `?:`. Trust -Wreorder, not -Wuninitialized, to catch this bug reliably.

Partially Fixed: One Member Down

```
capacity_(count > 0 ? count : 2),  
size_(count > 0 ? capacity_ : 0),  
data_(count > 0 ? new int[count]{} : new int[2]{})
```

- `data_` no longer references `capacity_` at all — it recomputes the same two cases directly from `count`.

This part is now correct, regardless of order.

- `size_` *still* reads `capacity_` before it's initialized — still broken.

Still flagged

The initializer-list order still doesn't match declaration order, so `-Wreorder` still fires — even though only one member is actually broken now.

Lesson for this constructor: keep each initializer independent — depend only on the constructor's parameters, not on another member.

Can We Use This?

```
MiniVector::MiniVector(std::size_t count) {  
    if (count > 0) {  
        data_ = new int[count];  
        for (int i = 0; i < count; i++) data_[i] = 0;  
        capacity_ = size_ = count;  
    } else {  
        capacity_ = 2; data_ = new int[capacity_]; size_ = 0;  
    }  
}
```

Yes — correct

Every assignment lives in the constructor *body*, run in *written* order. Compiles and runs clean under `-fsanitize=address,undefined`.

One warning to fix

`int i` vs. `std::size_t count` triggers `-Wsign-compare` (`-Wextra`) — use `std::size_t i`.